

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

ИНСТИТУТ НЕПРЕРЫВНОГО И ДИСТАНЦИОННОГО ОБРАЗОВАНИЯ

КАФЕДРА №3

ОЦЕНКА

ПРЕПОДАВАТЕЛЬ

старший преподаватель
должность, уч. степень, звание

подпись, дата

Н.А. Соловьева
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №6
Вариант 14

по дисциплине: Технология программирования

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

В3441
номер группы

подпись, дата

Р.С. Кулаков
инициалы, фамилия

Санкт-Петербург 2025

1 Задание

Б) Длительность выполнения задач из задания 1 записать в контейнер по варианту (вектор, список или словарь). Из всех элементов вычесть такое число, чтобы в наборе данных появились отрицательные значения. Для подготовленного таким образом набора данных выполнить действия из задания 2, используя функции из раздела `algorithm`.

2 Вариант задания

Кулаков Роман Станиславович	B3441	14	5	14	vector
-----------------------------	-------	----	---	----	--------

Вариант 14

В одномерном массиве, состоящем из n вещественных элементов, вычислить:

1. количество элементов массива, равных 0;
2. сумму элементов массива, расположенных после минимального элемента.

Упорядочить элементы массива по возрастанию модулей элементов.

3 Перечисление использованных методов контейнеров с пояснением результатов их работы

3.1 `push_back(value)`

Метод добавляет элемент в конец вектора. Используется для поэлементного наполнения вектора `durations` случайно сгенерированными значениями.

3.2 `begin()` и `end()`

Методы возвращают итераторы на начало и конец вектора соответственно. Эти итераторы используются как границы для функций из стандартной библиотеки алгоритмов (`transform`, `count`, `min_element`, `accumulate`, `sort`).

3.3 Диапазонный цикл for (тип переменная : контейнер)

Позволяет пройти по всем элементам вектора без использования индексов.

4 Перечисление использованных функции из раздела algorithm с пояснением результатов их работы

4.1 transform(begin, end, result_begin, функция)

Позволяет применить заданную функцию к каждому элементу диапазона и сохранить результат в другой диапазон. Применяется для вычитания фиксированного значения из всех элементов вектора.

4.2 count(begin, end, значение)

Подсчитывает количество вхождений указанного значения в заданном диапазоне. Используется для подсчета количества нулевых значений в векторе.

4.3 min_element(begin, end)

Находит итератор на минимальный элемент в заданном диапазоне. Применяется в программе для определения позиции минимального элемента, чтобы далее вычислить сумму всех элементов после него.

4.4 accumulate(begin, end, начальное_значение)

Вычисляет сумму всех элементов в диапазоне, начиная с заданного начального значения. Подсчет суммы элементов, следующих после минимального.

4.5 sort(begin, end, функция_сравнения)

Сортирует элементы диапазона по заданному критерию.

5 Скриншоты с примерами работы

```
Сгенерированные длительности:
0 70.0481 -44.2538 -77.1863 -30.5663 46.1399 1.37532 -3.22169 -72.5201 -0

Кол-во нулей: 2
Сумма после минимального: -58.7928
Отсортированный по модулю вектор: 0 -0 1.37532 -3.22169 -30.5663 -44.2538 46.1399 70.0481 -72.5201 -77.1863
```

```
Сгенерированные длительности:
-54.5172 65.3207 -24.725 20.6996 0 51.4623 7.52785 0.859554 -19.247 -12.4627

Кол-во нулей: 1
Сумма после минимального: 89.4354
Отсортированный по модулю вектор: 0 0.859554 7.52785 -12.4627 -19.247 20.6996 -24.725 51.4623 -54.5172 65.3207
```

```
Сгенерированные длительности:
54.6759 -40.2864 76.7011 -20.5013 4.7467 1.9512 -3.08344 -56.7664 32.2845 16.9398

Кол-во нулей: 0
Сумма после минимального: 49.2242
Отсортированный по модулю вектор: 1.9512 -3.08344 4.7467 16.9398 -20.5013 32.2845 -40.2864 54.6759 -56.7664 76.7011
```

6 Текст программы

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <numeric>
#include <cmath>
#include <ctime>

using namespace std;

int main() {
    setlocale(LC_ALL, "Russian");
    std::srand(std::time(nullptr));

    // генерация данных из задания 1, поскольку оно не выполняется
    vector<double> durations;
    const int N = 10;
    for (int i = 0; i < N; ++i) {
        double r = (std::rand() / (double)RAND_MAX) * 100.0 - 5.0;
        // часть элементов специально 0
        if (std::rand() % 10 == 0) r = 0.0;
        // часть элементов специально -
        if (std::rand() % 2 == 0) r *= -1;
        durations.push_back(r);
    }

    // вычитаем константу, чтобы появились отрицательные
    double shift = 0;
    transform(durations.begin(), durations.end(), durations.begin(),
        [shift](double x) { return x - shift; });

    cout << "Сгенерированные длительности:\n";
    for (double d : durations) cout << d << " ";
    cout << "\n\n";

    // нули
    int zeroCount = count(durations.begin(), durations.end(), 0.0);
```

```
// сумма элементов после минимального
auto minIt = min_element(durations.begin(), durations.end());
double sumAfterMin = accumulate(minIt + 1, durations.end(), 0.0);

// сортировка по возрастанию модулей
sort(durations.begin(), durations.end(),
    [](double a, double b) {
        return abs(a) < abs(b);
    });

cout << "Кол-во нулей: " << zeroCount << endl;
cout << "Сумма после минимального: " << sumAfterMin << endl;

cout << "Отсортированный по модулю вектор: ";
for (double d : durations) {
    cout << d << " ";
}
cout << endl;

return 0;
}
```